

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ
BÖLÜMÜ**



BLM 3522 PROJE RAPORU

Projenin Başlığı

İot ve Akıllı Şehir Uygulaması

Öğrencinin Adı Soyadı

Erkim Berk

Ünsal

Öğrenci

Numarası

23291218

Proje Raporu

Ad Soyad	Erkim Berk Ünsal
Öğrenci Numarası	23291218
Ders	BLM3522 – Bulut Bilişim ve Uygulamaları
Proje No	Proje 4
Proje Konusu	IoT ve Akıllı Şehir Uygulaması
Bulut Platformu	Amazon Web Services (AWS)
Tarih	15.06.2026
GitHub Repo	https://github.com/erkimberk/BLM3522
Video Linki	https://youtu.be/AFyIfMRpwLw

1. Giriş ve Projenin Amacı

Akıllı şehir kavramı, bir şehirdeki çeşitli altyapı bileşenlerinin sensörler aracılığıyla izlenmesi ve toplanan verinin merkezi olarak işlenip yönetilmesi anlamına gelir. Trafik akışından hava kalitesine, atık yönetiminden enerji tüketimine kadar pek çok alan bu kapsamda değerlendirilebilir. Bu tür sistemlerin temelinde, çok sayıda cihazdan sürekli veri akışını toplayabilecek ve ölçeklenebilir şekilde işleyebilecek bir altyapı ihtiyacı vardır. Bulut bilişim, bu ihtiyaca cevap veren en uygun çözümlerden biridir.

Bu projede AWS bulut platformu üzerinde çalışan bir IoT akıllı şehir uygulaması geliştirdim. Projenin amacı, bir şehirdeki sensörlerden veri toplayıp bu veriyi buluta göndermek, orada işleyip saklamak ve gerçek zamanlı olarak bir web panelinde görselleştirmektir. Böylece hem IoT cihazlarıyla bulut servislerinin nasıl bütünleştiğini hem de toplanan veriden nasıl anlamlı bilgi çıkarıldığını uygulamalı olarak öğrenmeyi hedefledim.

Uygulamada üç farklı sensör tipini ele aldım: trafik yoğunluğunu ölçen trafik ışığı sensörü, hava kirliliğini ölçen hava kalitesi sensörü ve doluluk oranını ölçen çöp kutusu sensörü. Fiziksel donanım kullanmadan sistemi geliştirip test edebilmek için bu sensörleri Python ile sanal olarak ürettim. Bu yaklaşım, gerçek bir donanım olmadan da bulut tarafındaki tüm akışın doğru çalıştığını doğrulamama imkan verdi.

2. Kullanılan Teknolojiler

Projede kullandığım başlıca teknolojiler ve bunların projedeki rolleri aşağıda özetlenmiştir.

Teknoloji	Projedeki Rolü
Python 3.11	Sensör simülasyonu, MQTT istemcisi ve REST API geliştirme
AWS IoT Core	Cihazlardan MQTT ile veri toplayan bulut servisi
MQTT (TLS)	Sensör verisinin güvenli şekilde iletilmesi
AWS Lambda	Gelen veriyi işleyen sunucusuz fonksiyon
AWS DynamoDB	Ölçümlerin ve uyarıların saklandığı NoSQL veritabanı
Flask / Flask-CORS	Veriyi sunan REST API
Chart.js	Web panelinde grafik görselleştirme
paho-mqtt, boto3	MQTT bağlantısı ve AWS servis erişimi kütüphaneleri

3. Sistem Mimarisi

Sistemde verinin izlediği yol şu şekildedir:

Sensör → MQTT → AWS IoT Core → Rule Engine → Lambda → DynamoDB → REST API → Dashboard

Python ile yazılan sensör simülatörü gerçekçi ölçüm verileri üretir. Bu veri, MQTT protokolü ve TLS sertifikaları kullanılarak AWS IoT Core'a yayınlanır. IoT Core üzerindeki Rule Engine, belirlenen topic desenine gelen mesajları yakalayıp bir Lambda fonksiyonunu tetikler. Lambda fonksiyonu veriyi

DynamoDB'ye kaydeder ve eşik değerleri aşan durumlar için otomatik uyarı oluşturur. Son olarak Flask ile yazılan REST API bu veriyi dışarıya JSON olarak sunar ve web panelinde grafiklerle gösterilir.

Bu mimarinin önemli bir avantajı, bileşenlerin birbirinden bağımsız çalışmasıdır. Sensör sayısı arttığında IoT Core ve Lambda otomatik olarak ölçeklenir; veritabanı tarafında ayrı bir sunucu yönetimi gerekmez. Sunucusuz (serverless) yaklaşım sayesinde yalnızca işlenen veri kadar kaynak kullanılır, bu da maliyet açısından verimlidir.

4. Proje Bileşenleri

4.1. Sensör Simülatörleri

src/devices klasöründe üç sensör sınıfı bulunmaktadır. Her sınıfın generate_data fonksiyonu çağrıldığında gerçekçi bir ölçüm verisi üretir. Sensörler tamamen rastgele değil, mantıksal kurallara göre veri üretecek şekilde tasarlanmıştır; örneğin trafik yoğunluğu arttığında ortalama araç hızı düşer.

- traffic_light_sensor.py – Trafik yoğunluğu, geçen araç sayısı ve ortalama hız üretir. Yoğun saat (peak hour) aktif olduğunda yoğunluğu artırır ve duruma göre yeşil, sarı veya kırmızı durumu belirler.
- air_quality_sensor.py – PM2.5, PM10, CO2, sıcaklık ve nem değerleri üretir. PM2.5 değerinden, dünya standartlarına dayalı bir formülle hava kalitesi indeksini (AQI) hesaplar ve Good, Fair, Moderate gibi kategorilere ayırır.
- trash_bin_sensor.py – Çöp kutusunun zamanla dolmasını simüle eder, doluluk yüzdesini ve ağırlığını hesaplar. Doluluk %85'i geçtiğinde boşaltma gerektiğini bildirir ve aciliyet seviyesini belirler.

4.2. MQTT İstemcisi ve AWS Yapılandırması

src/utlis klasöründeki aws_config.py modülü tüm yapılandırmayı .env dosyasından okur. AWS bölgesi, IoT endpoint adresi, MQTT portu, topic ön eki ve sertifika yolları bu dosyada merkezi olarak tutulur. Hassas bilgilerin koda gömülmemesi güvenlik açısından önemli bir tercihtir.

mqtt_client.py modülü ise paho-mqtt kütüphanesini sarmalayarak AWS IoT Core'a bağlanmayı yönetir. Bağlantı, kullanıcı adı ve şifre yerine cihaz sertifikası ve özel anahtar kullanılarak, TLS ile şifreli biçimde kurulur. Veriler QoS seviyesi 1 ile, yani en az bir kez teslim garantisiyle yayınlanır. Modül ayrıca bağlantı kopması ve sertifika hatası gibi durumlar için anlamlı log mesajları üretir.

4.3. MQTT Topic Yapısı

Sensör verileri, hiyerarşik bir topic yapısıyla yayınlanır. Bu yapı, IoT Core tarafında mesajların kolayca filtrelenmesini sağlar.

```
smart-city/{sensor_tipi}/{cihaz_id}/{metrik}
```

Örnek: smart-city/traffic_light/traffic-light-001/congestion. Rule Engine kuralı, smart-city/+/+/+ deseni ile tüm sensör mesajlarını tek bir kuralda yakalayabilir.

4.4. Lambda Fonksiyonu

src/lambda/iot_to_dynamodb.py dosyası, IoT Core'dan mesaj geldiğinde otomatik olarak tetiklenir. Fonksiyon üç temel işlem yapar: gelen mesajı çözümler, veriyi DynamoDB'ye kaydeder ve eşik değeri kontrolü yaparak gerekirse uyarı oluşturur.

Eşik değerleri ve buna bağlı uyarı mantığı aşağıdaki gibidir:

Sensör	Metrik	Eşik Değer	Aşıldığında
Trafik Işığı	congestion	85	Yüksek yoğunluk uyarısı
Hava Kalitesi	pm25	75	Kötü hava kalitesi uyarısı
Çöp Kutusu	fill_level	90	Boşaltma uyarısı

Uyarının ciddiyeti, ölçülen değerin eşik değeri olan uzaklığına göre belirlenir: eşik değeri %50 üstündeyse critical, %20 üstündeyse high, daha azsa medium olarak işaretlenir. Uyarılar ayrı bir tabloya kaydedilerek panelde öncelik sırasına göre gösterilir.

4.5. DynamoDB Tabloları

Veriler iki tabloda saklanır. sensor-readings tablosu tüm ölçümleri, sensor-alerts tablosu ise uyarıları tutar. Tablo tasarımında partition key olarak sensör tipi ve cihaz kimliğinin birleşimi, sort key olarak ise zaman damgası kullanılır. Bu yapı, belirli bir cihazın belirli bir zaman aralığındaki verilerini hızlıca sorgulamaya olanak tanır.

Ayrıca her kayda 30 günlük bir TTL (Time To Live) değeri eklenir. Bu sayede eski kayıtlar otomatik olarak silinir; veritabanı sürekli büyümeyen ve depolama maliyeti kontrol altında tutulur. Sensör tipine ve lokasyona göre sorgu yapabilmek için ikincil indeksler (GSI) tanımlanmıştır.

4.6. REST API ve Dashboard

src/api/app.py dosyasındaki Flask uygulaması, DynamoDB'deki veriyi JSON olarak sunan endpoint'ler içerir. Flask-CORS eklenerek web panelinin API'ye sorunsuz erişmesi sağlanmıştır. frontend klasöründeki dashboard ise bu endpoint'lerden veri çekip Chart.js grafikleriyle gösterir ve belirli aralıklarla kendini yenileyerek gerçek zamanlı bir görünüm sunar.

API tarafındaki başlıca endpoint'ler:

- /api/health – Sistemin ve veritabanı bağlantısının durumunu kontrol eder.
- /api/sensors/<tür> – Belirtilen sensör tipinin son ölçümlerini döndürür.
- /api/sensors/<tür>/<cihaz> – Bir cihazın geçmiş verilerini ve özet istatistiklerini verir.
- /api/alerts – Aktif uyarıları ciddiyet sırasına göre döndürür.
- /api/stats/<tür>/<cihaz> – Son 24 saatlik minimum, maksimum ve ortalama değerleri hesaplar.

5. Güvenlik

Projede güvenlik birkaç katmanda ele alınmıştır. İlk olarak, sensörlerle AWS IoT Core arasındaki tüm iletişim TLS ile şifrelenir ve cihazlar sertifika tabanlı kimlik doğrulamasıyla bağlanır. Bu, yetkisiz cihazların sisteme veri göndermesini engeller.

İkinci olarak, IoT cihazına atanan AWS IoT politikası (iot_policy.json) en az yetki prensibine göre hazırlanmıştır. Cihaza yalnızca smart-city topic'lerine bağlanma, yayınlama ve abone olma izni verilir; gereksiz hiçbir yetki tanımlanmaz. Üçüncü olarak, AWS erişim anahtarları ve endpoint bilgileri koda gömülmek yerine .env dosyasında tutulur ve bu dosya .gitignore ile sürüm kontrolünün dışında bırakılır.

6. Karşılaşılan Zorluklar ve Çözümler

Geliştirme sürecinde karşılaştığım başlıca zorluklar ve bunları nasıl çözdüğüm aşağıda özetlenmiştir.

- Sertifika ayarları: AWS IoT Core'a ilk bağlantı denemelerinde TLS el sıkışma hataları aldım. Sorunun sertifika dosya yollarının yanlış olmasından kaynaklandığını tespit edip, bağlantı öncesinde dosyaların varlığını kontrol eden bir doğrulama ekledim.
- MQTT callback sürümü: paho-mqtt kütüphanesinin yeni sürümünde callback imzaları değişmişti; istemciyi VERSION2 callback API'si ile yapılandırarak uyumluluğu sağladım.
- Veri olmadan test: Gerçek bir IoT cihazı olmadığı için sistemi test etmek zordu. Bunu, gerçekçi veri üreten sensör simülatörleri yazarak çözdüm ve tüm akışı uçtan uca doğruladım.
- Veritabanı maliyeti: Sürekli veri akışının tabloyu şişirmesini önlemek için kayıtlara TTL ekleyerek eski verilerin otomatik silinmesini sağladım.

7. Çalıştırma Adımları

Proje aşağıdaki adımlarla çalıştırılır. Öncelikle sanal ortam oluşturulup bağımlılıklar kurulur:

```
python -m venv venv venv\Scripts\activate pip install -r requirements.txt
```

Ardından .env dosyasına AWS bilgileri girilir ve sensör simülatörü başlatılır:

```
python sensor_simulator.py --duration 300 --interval 5
```

Bu komut 300 saniye boyunca her 5 saniyede bir üç sensörden de veri üretip AWS IoT Core'a gönderir. REST API ayrı bir terminalde başlatılır:

```
python src/api/app.py
```

Son olarak tarayıcıdan panele erişilerek veriler gerçek zamanlı izlenir. Panelde sensör değerleri, grafikler ve eşik aşımı durumunda oluşan uyarılar görüntülenir.

8. Sonuç ve Değerlendirme

Bu proje sayesinde IoT cihazlarından veri toplama, MQTT protokolü ile buluta güvenli veri gönderme, AWS IoT Core, Lambda ve DynamoDB servislerini birlikte kullanma ve verileri gerçek zamanlı görselleştirme konularında uygulamalı deneyim kazandım. Sunucusuz mimarinin nasıl kurulduğunu ve bileşenlerin birbirleriyle nasıl haberleştiğini bizzat görerek öğrendim.

Geliştirilen sistem; trafik, hava kalitesi ve çöp yönetimini tek bir merkezi panelden izlenebilir hale getirerek temel bir akıllı şehir senaryosunu başarıyla uygulamaktadır. Mimari, daha fazla sensör tipi ve cihaz eklenecek şekilde genişletilebilir niteliktedir. İleride, toplanan veriler üzerinde makine öğrenmesi tabanlı tahmin (örneğin trafik yoğunluğu öngörüsü) veya harita üzerinde konum bazlı görselleştirme gibi geliştirmeler eklenebilir.

9. Baęlantılar

GitHub: <https://github.com/erkimberk/BLM3522>

Video: <https://youtu.be/AFylfMRpwLw>